WIPRO SDET CAPSTONE

Playwright Notes

Beginner to Viva Ready

Demo Web Shop automation project using Playwright, JavaScript, Page Object Model, Allure Report, and GitHub Pages.

Item	Value
Website under test	https://demowebshop.tricentis.com
Automation tool	Playwright with JavaScript
Design pattern	Page Object Model plus reusable fixtures
Functional coverage	6 core modules + 2 add-on modules, each with 15 test cases = 120 test cases
Browser coverage	Chromium, Firefox, WebKit
Total executions	120 tests x 3 browsers = 360 executions
Report	Allure HTML report, hosted from the gh-pages branch
Local project	D:\WIPRO\Capstone_Project
Generated on	02 June 2026

How to study this PDF

Read it in this order: first understand the project goal, then the folder structure, then the Playwright basics, then the Page Object Model, then one complete test flow, and finally the viva questions. The idea is simple: every spec file calls one page-object method, and that method performs browser actions and assertions on Demo Web Shop.

Study Roadmap

- Project snapshot and capstone requirement
- Testing basics in simple language
- Playwright basics with JavaScript
- Complete folder and file structure
- Configuration, dependencies, and scripts
- Page Object Model and fixtures
- How one test case runs from command to report
- Service-wise coverage and all 120 test cases
- Allure report and GitHub Actions deployment
- Debugging, troubleshooting, and viva preparation

One-line project explanation for viva

This project automates 120 functional test cases for the Demo Web Shop e-commerce website using Playwright JavaScript. The tests are organized service-wise with Page Object Model classes, run across Chromium, Firefox, and WebKit, and publish an Allure report through GitHub Pages.

1. Testing Basics From Zero

Before Playwright, understand what we are trying to prove. A tester does not just click randomly. A tester checks whether a feature behaves as expected under a known condition.

Term	Meaning in simple words	Example from this project
AUT	Application Under Test. The website we are testing.	Demo Web Shop
Test case	A small check with a clear expected result.	Login page should show email and password fields.
Assertion	The proof line in automation. If it fails, the test fails.	expect(locator).toBeVisible()
Regression	Running old tests again to make sure new changes did not break them.	Run all 120 tests before final submission.
Cross-browser	Same tests in different browser engines.	Chromium, Firefox, WebKit
Report	Readable result summary after execution.	Allure report hosted on GitHub Pages

Why This Website Was Selected

- Demo Web Shop has e-commerce flows similar to a real application: login, products, cart, account pages, checkout-related pages, customer information, wishlist, and shipping information.
- It does not require us to expose ourselves as developers or build a custom website.
- It is more stable for automation than a site that repeatedly asks for Cloudflare verification.
- It gives enough independent services to satisfy the capstone requirement of 6 core modules and 2 add-on modules and minimum 120 test cases.

2. Playwright Basics

Playwright is an automation framework. It launches a browser, opens pages, finds elements, performs actions, and checks expected results. In this project we use the Playwright test runner with JavaScript.

Concept	What it does	Project example
test()	Creates one automated test case.	AUTH-001 Login page opens.
test.describe()	Groups related test cases.	Authentication Module
page	Browser tab controlled by Playwright.	page.goto('/login') inside BasePage.navigate()
locator	Finds an element reliably.	page.locator('#Email')
expect	Assertion library.	await expect(locator).toBeVisible()
project	Browser configuration in Playwright.	chromium, firefox, webkit
reporter	Creates result output.	html and allure-playwright
fixture	Reusable setup object passed to tests.	baseFixture.js creates page objects

Important Commands

Command	When to use it	What you should expect
npm install	First time after cloning/downloading the project.	Installs Playwright, Allure adapter, and project dependencies.
npx playwright test	Main command to run the complete suite.	Runs all specs from tests/ on all configured browsers.
npx playwright test --project=chromium	When you want only one browser locally.	Runs 120 test cases on Chromium only.
npx playwright test tests/authentication	When checking one service.	Runs only Authentication specs.
npx playwright test --list	Before running, to see what Playwright found.	Lists tests and browser projects.
npx playwright show-report	After a local run with HTML reporter.	Opens the Playwright HTML

		report.
npx allure generate allure-results --clean -o allure-report	After tests when you need Allure locally.	Builds static Allure HTML files.
npx allure open allure-report	To view local Allure report.	Starts a local web server and opens Allure.

3. Project Structure

A Playwright engineer keeps test code separated by responsibility. The test files should read like scenarios, while page objects contain selectors, navigation, and assertions.

```
Capstone_Project/
|-- .github/
|   |-- workflows/allure-report.yml
|-- api/
|   |-- apiClient.js
|-- data/
|   |-- demoWebShopData.js
|-- docs/
|   |-- Demo_Web_Shop_Capstone_Day_Wise_Testing_Plan.docx
|   |-- Demo_Web_Shop_Capstone_Day_Wise_Testing_Plan.pdf
|   |-- Demo_Web_Shop_Playwright_Capstone_Final_Report.docx
|   |-- Demo_Web_Shop_Playwright_Capstone_Final_Report.pdf
|   |-- Playwright_Notes.pdf
|-- fixtures/
|   |-- baseFixture.js
|-- pages/
|   |-- addressShipping.page.js
|   |-- authentication.page.js
|   |-- base.page.js
|   |-- cart.page.js
|   |-- checkoutPayment.page.js
|   |-- customerSupport.page.js
|   |-- product.page.js
|   |-- userProfile.page.js
|   |-- wishlistCompare.page.js
|-- tests/
|   |-- authentication/ (15 spec files)
|   |   |-- auth-001-login-page-opens-with-correct-title-and-heading.spec.js
|   |   |-- auth-002-login-page-shows-email-and-password-fields.spec.js
|   |   |-- auth-003-login-page-shows-login-button.spec.js
|   |   |-- ...
|   |-- product/ (15 spec files)
|   |   |-- prod-001-home-page-shows-welcome-content-and-featured-products.spec.js
|   |   |-- prod-002-top-navigation-exposes-catalog-categories.spec.js
|   |   |-- prod-003-books-category-shows-product-cards.spec.js
|   |   |-- ...
|   |-- cart/ (15 spec files)
|   |   |-- cart-001-empty-cart-page-opens.spec.js
|   |   |-- cart-002-empty-cart-message-is-shown.spec.js
```

```
|   |   |-- cart-003-header-shopping-cart-link-is-visible.spec.js
|   |   |-- ...
|   |-- wishlist-compare/ (15 spec files)
|   |   |-- wish-001-wishlist-page-opens.spec.js
|   |   |-- wish-002-empty-wishlist-message-is-shown.spec.js
|   |   |-- wish-003-header-wishlist-link-is-visible.spec.js
|   |   |-- ...
|   |-- user-profile/ (15 spec files)
|   |   |-- user-001-my-account-page-requires-login-for-guest.spec.js
|   |   |-- user-002-order-history-page-requires-login-for-guest.spec.js
|   |   |-- user-003-addresses-page-requires-login-for-guest.spec.js
|   |   |-- ...
|   |-- address-shipping/ (15 spec files)
|   |   |-- addr-001-shipping-returns-page-opens.spec.js
|   |   |-- addr-002-shipping-returns-page-has-customer-service-context.spec.js
|   |   |-- addr-003-guest-addresses-page-redirects-to-login.spec.js
|   |   |-- ...
|   |-- add-on-checkout-payment/ (15 spec files)
|   |   |-- add-on-pay-001-empty-cart-keeps-user-on-shopping-cart-page.spec.js
|   |   |-- add-on-pay-002-empty-cart-blocks-checkout-with-empty-cart-message.spec.js
|   |   |-- add-on-pay-003-cart-page-is-available-before-checkout.spec.js
|   |   |-- ...
|   |-- add-on-customer-support/ (15 spec files)
|   |   |-- add-on-info-001-contact-us-page-opens.spec.js
|   |   |-- add-on-info-002-contact-us-page-has-full-name-field.spec.js
|   |   |-- add-on-info-003-contact-us-page-has-email-field.spec.js
|   |   |-- ...
|-- .gitignore
|-- package.json
|-- package-lock.json
|-- playwright.config.js
|-- README.md
|-- node_modules/ (installed dependencies, not written by us)
|-- allure-results/ (generated test result data)
|-- allure-report/ (generated HTML report when created locally)
|-- playwright-report/ (generated Playwright HTML report)
|-- test-results/ (generated traces/screenshots/videos on failure)
```

File and Folder Purpose

Path	Why it exists
tests/	Contains the actual spec files. Each service has 15 spec files.
pages/	Contains Page Object Model classes. Each class knows how to test one service area.
pages/base.page.js	Common browser and assertion helper methods used by all page objects.
fixtures/baseFixture.js	Creates reusable page-object fixtures so future specs can ask for authenticationPage, productPage, etc.

data/demoWebShopData.js	Stores project-level data such as service coverage, browser list, and expected execution counts.
api/apiClient.js	Small API helper showing how internal API requests can be wrapped if API tests are added.
playwright.config.js	Central Playwright configuration: test folder, browsers, timeouts, reporters, base URL, retry behavior.
package.json	Defines project name, dependencies, and runnable npm scripts.
.github/workflows/allure-report.yml	CI/CD workflow that runs tests and publishes the Allure report to gh-pages.
.gitignore	Prevents generated folders such as node_modules, allure-report, and test-results from being committed.
docs/	Contains project documents and this notes PDF.
allure-results/	Generated raw Allure result JSON files after a test run.
allure-report/	Generated static Allure website after running allure generate.
playwright-report/	Generated Playwright HTML report.
test-results/	Generated failure assets such as trace, screenshots, and videos.

4. Dependencies, Scripts, and Configuration

package.json

package.json is the project identity card. It tells Node.js what this project is, which packages it needs, and which commands are available.

Field	Explanation
Project name	demo-web-shop-playwright-capstone
Module system	module means JavaScript ES module import/export syntax is used.
Main test framework	@playwright/test
Reporting tools	HTML reporter and Allure reporter
Important scripts	test, test:headed, test:list, report, allure:generate, allure:open, test:allure

```
{
  "name": "demo-web-shop-playwright-capstone",
  "version": "1.0.0",
  "description": "Playwright JavaScript automation capstone for the Demo Web Shop e-commerce site.",
  "type": "module",
  "scripts": {
    "test": "playwright test",
    "test:headed": "playwright test --headed",
    "test:list": "playwright test --list",
```

```
    "report": "playwright show-report",
    "allure:generate": "allure generate allure-results --clean -o allure-report",
    "allure:open": "allure open allure-report",
    "test:allure": "npm run test && npm run allure:generate"
  },
  "devDependencies": {
    "@playwright/test": "^1.60.0",
    "@types/node": "^25.9.1",
    "allure-commandline": "^2.41.0",
    "allure-playwright": "^3.9.0"
  }
}
```

playwright.config.js

This file controls how Playwright behaves. When you run npx playwright test, Playwright reads this configuration first.

Setting	Meaning
testDir: './tests'	Playwright searches for spec files inside the tests folder.
fullyParallel: false	Keeps execution controlled and easier to debug for training/capstone work.
forbidOnly: !!process.env.CI	Stops accidental test.only from reaching GitHub Actions.
retries: process.env.CI ? 1 : 0	Retries once in CI but not locally.
workers: 1	Runs one worker at a time, which is simpler and stable for this website.
timeout: 90000	Gives each test up to 90 seconds.
baseURL	Lets page.goto('/login') resolve against Demo Web Shop.
reporter	Creates both Playwright HTML report and Allure results.
projects	Runs same tests in Chromium, Firefox, and WebKit.

```
// @ts-check
import { defineConfig, devices } from '@playwright/test';


export default defineConfig({
  testDir: './tests',
  fullyParallel: false,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 1 : 0,
  workers: 1,
  timeout: 90000,
  expect: {
    timeout: 30000,
  },
  reporter: [
    ['html', { open: 'never' }],
```

```
    ['allure-playwright', { resultsDir: 'allure-results' }],
  ],
  use: {
    baseURL: 'https://demowebshop.tricentis.com',
    actionTimeout: 20000,
    navigationTimeout: 60000,
    screenshot: 'only-on-failure',
    trace: 'retain-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
  ],
});
```

5. Page Object Model and Fixtures

Page Object Model, usually called POM, means we keep browser actions and assertions inside page classes instead of repeating them inside every spec file. This makes the suite easier to read, maintain, and explain.

Without POM	With POM in this project
Every spec repeats navigation, locators, and expect statements.	Spec file calls one meaningful method such as verifyAUTH001LoginPageOpensWithCorrectTitleAndHeading().
Changing a selector requires editing many files.	Common logic lives in the page object, so related changes stay in one place.
Test code becomes long and hard to explain.	Spec files stay short and page object methods explain the behavior.

BasePage

BasePage is the parent class. All service page classes extend it, so they inherit reusable actions and assertions.

```
// @ts-check
import { expect } from '@playwright/test';


export class BasePage {
  constructor(page) {
    this.page = page;
  }


  async navigate(route) {
    await this.page.goto(route, { waitUntil: 'domcontentloaded' });
  }


  async verifyTitle(value) {
    await expect(this.page).toHaveTitle(value);
  }


  async verifyUrl(value) {
    await expect(this.page).toHaveURL(value);
  }


  async verifyHeading(value) {
    await expect(this.page.locator('h1').first()).toContainText(value);
  }


  async verifyBodyText(value) {
    await expect(this.page.locator('body')).toContainText(value);
  }


  async verifyVisible(selector) {
    await expect(this.page.locator(selector).first()).toBeVisible();
  }


  async verifyEnabled(selector) {
    await expect(this.page.locator(selector).first()).toBeEnabled();
  }


  async verifyChecked(selector) {
    await expect(this.page.locator(selector).first()).toBeChecked();
  }


  async verifyValue(selector, value) {
    await expect(this.page.locator(selector).first()).toHaveValue(value);
  }


  async verifyContainsText(selector, value) {
    await expect(this.page.locator(selector).first()).toContainText(value);
  }


  async verifyCountAtLeast(selector, value) {
```

```
    const count = await this.page.locator(selector).count();
    expect(count).toBeGreaterThanOrEqual(value);
  }
}
```

baseFixture.js

Fixtures are reusable objects created by Playwright before the test needs them. The current specs mostly instantiate page objects directly, but this fixture file shows the professional scalable pattern for the same project.

```
// @ts-check
import { test as base, expect } from '@playwright/test';
import { AuthenticationPage } from '../pages/authentication.page.js';
import { ProductPage } from '../pages/product.page.js';
import { CartPage } from '../pages/cart.page.js';
import { WishlistComparePage } from '../pages/wishlistCompare.page.js';
import { UserProfilePage } from '../pages/userProfile.page.js';
import { AddressShippingPage } from '../pages/addressShipping.page.js';
import { CheckoutPaymentPage } from '../pages/checkoutPayment.page.js';
import { CustomerSupportPage } from '../pages/customerSupport.page.js';

export const test = base.extend({
  authenticationPage: async ({ page }, use) => {
    await use(new AuthenticationPage(page));
  },
  productPage: async ({ page }, use) => {
    await use(new ProductPage(page));
  },
  cartPage: async ({ page }, use) => {
    await use(new CartPage(page));
  },
  wishlistComparePage: async ({ page }, use) => {
    await use(new WishlistComparePage(page));
  },
  userProfilePage: async ({ page }, use) => {
    await use(new UserProfilePage(page));
  },
  addressShippingPage: async ({ page }, use) => {
    await use(new AddressShippingPage(page));
  },
  checkoutPaymentPage: async ({ page }, use) => {
    await use(new CheckoutPaymentPage(page));
  },
  customerSupportPage: async ({ page }, use) => {
    await use(new CustomerSupportPage(page));
  },
});

export { expect };
```

Beginner translation

A spec file is like the question paper. A page object is like the answer method. BasePage is the common toolbox. Fixtures are a clean way to give that toolbox and the page objects to tests automatically.

6. How One Test Runs

Let us follow AUTH-001 from command to result. This same pattern is repeated for the whole suite.

1. You run npx playwright test.
2. Playwright reads playwright.config.js.
3. It finds spec files under tests/.
4. For each browser project, Playwright launches that browser.
5. The spec file creates a page object in beforeEach.
6. The test calls one page-object verification method.
7. The page-object method uses BasePage helpers to navigate and assert.
8. If all assertions pass, Playwright marks the test passed.
9. The HTML reporter writes playwright-report.
10. The Allure reporter writes raw results into allure-results.
11. GitHub Actions later generates allure-report and deploys it to gh-pages.

AUTH-001 Spec File

```
// @ts-check
import { test } from '@playwright/test';
import { AuthenticationPage } from '../../pages/authentication.page.js';


test.describe('Authentication Module', () => {
  let authenticationPage;


  test.beforeEach(async ({ page }) => {
    authenticationPage = new AuthenticationPage(page);
  });


  test('AUTH-001 Login page opens with correct title and heading', async () => {
    await authenticationPage.verifyAUTH001LoginPageOpensWithCorrectTitleAndHeading();
  });
});
```

AUTH-001 Page Object Method

```
async verifyAUTH001LoginPageOpensWithCorrectTitleAndHeading() {
    await this.navigate("/login");
    await this.verifyTitle(/Login/);
    await this.verifyHeading("Welcome, Please Sign In!");
  }
```

What each line means

Line	Meaning
import { test } from '@playwright/test'	Imports Playwright test runner.
import { AuthenticationPage }	Imports the page object class for authentication tests.
test.describe(...)	Groups this test under Authentication Module.
test.beforeEach(...)	Creates a fresh AuthenticationPage object before the test.
test('AUTH-001 ...')	Defines the actual test case name shown in report.
authenticationPage.verifyAUTH001...	Calls the business-level method that performs browser checks.
this.navigate('/login')	Opens the login page using baseURL from config.
this.verifyTitle(/Login/)	Asserts browser title contains Login.
this.verifyHeading(...)	Asserts the visible page heading is correct.

7. Service Modules

The capstone requires 6 core modules, 2 add-on modules, and 15 test cases per module. This project covers exactly 6 core modules and 2 add-on modules, each with 15 functional tests.

Service	Test folder	Page object	Methods
Authentication	authentication	pages/authentication.page.js	15
Product and Search	product	pages/product.page.js	15
Cart	cart	pages/cart.page.js	15
Wishlist and Compare	wishlist-compare	pages/wishlistCompare.page.js	15
User Profile and Account	user-profile	pages/userProfile.page.js	15
Address and Shipping	address-shipping	pages/addressShipping.page.js	15
Add on - Checkout and Payment	checkout-payment	pages/checkoutPayment.page.js	15
Add on - Customer Support and	customer-support	pages/customerSupport.page.js	15

Information			

Authentication

Login page, register page, password recovery, protected account pages, and header login/register links.

Page-object method	Route hints	Assertion helpers used
verifyAUTH001LoginPageOpensWithCorrectTitleAndHeading	"/login"	verifyHeading, verifyTitle
verifyAUTH002LoginPageShowsEmailAndPasswordFields	"/login"	verifyVisible
verifyAUTH003LoginPageShowsLoginButton	"/login"	verifyEnabled, verifyVisible
verifyAUTH004LoginPageShowsRememberMeAndForgotPasswordOptions	"/login"	verifyBodyText, verifyVisible
verifyAUTH005LoginPageShowsRegistrationOptionForNewCustomer	"/login"	verifyBodyText, verifyVisible
verifyAUTH006PasswordRecoveryPageOpensCorrectly	"/passwordrecovery"	verifyHeading, verifyTitle
verifyAUTH007PasswordRecoveryPageHasEmailFieldAndRecoverButton	"/passwordrecovery"	verifyVisible
verifyAUTH008RegisterPageOpensCorrectly	"/register"	verifyHeading, verifyTitle
verifyAUTH009RegisterPageShowsGenderChoices	"/register"	verifyVisible
verifyAUTH010RegisterPageShowsPersonalDetailFields	"/register"	verifyVisible
verifyAUTH011RegisterPageShowsPasswordFields	"/register"	verifyVisible
verifyAUTH012RegisterButtonIsAvailable	"/register"	verifyEnabled, verifyVisible
verifyAUTH013ProtectedAccountPageRedirectsGuestUserToLogin	"/customer/info"	verifyHeading, verifyTitle
verifyAUTH014ProtectedOrdersPageRedirectsGuestUserToLogin	"/customer/orders"	verifyHeading, verifyTitle
verifyAUTH015HeaderExposesLoginAndRegisterLinks	"/"	verifyVisible

Product and Search

Home page, category pages, product cards, product detail pages, sorting controls, and search results.

Page-object method	Route hints	Assertion helpers used
verifyPROD001HomePageShowsWelcomeContentAndFeaturedProducts	"/"	verifyBodyText
verifyPROD002TopNavigationExposesCatalogCategories	"/"	verifyVisible
verifyPROD003BooksCategoryShowsProductCards	"/books"	verifyCountAtLeast, verifyHeading
verifyPROD004ComputersCategoryShowsSubcategories	"/computers"	verifyBodyText, verifyHeading
verifyPROD005ElectronicsCategoryShowsSubcategories	"/electronics"	verifyBodyText, verifyHeading
verifyPROD006ApparelAndShoesCategoryShowsProducts	"/apparel-shoes"	verifyCountAtLeast, verifyHeading
verifyPROD007DigitalDownloadsCategoryOpens	"/digital-downloads"	verifyCountAtLeast, verifyHeading

verifyPROD008JewelryCategoryOpensWithProductCards	"/jewelry"	verifyCountAtLeast, verifyHeading
verifyPROD009GiftCardsCategoryOpensWithProducts	"/gift-cards"	verifyCountAtLeast, verifyHeading
verifyPROD010BuildYourOwnComputerProductDetailsAreVisible	"/build-your-own-computer"	verifyHeading, verifyVisible
verifyPROD011LaptopProductDetailsAreVisible	"/141-inch-laptop"	verifyBodyText, verifyHeading
verifyPROD012SmartphoneProductDetailsAreVisible	"/smartphone"	verifyBodyText, verifyHeading
verifyPROD013BookProductPageShowsPriceAndShippingText	"/computing-and-internet"	verifyBodyText, verifyHeading
verifyPROD014CategoryPageHasSortAndPageSizeControls	"/books"	verifyVisible
verifyPROD015SearchPageReturnsComputerProducts	"/search?q=computer"	verifyBodyText, verifyHeading, verifyVisible

Cart

Shopping cart page, empty cart state, cart links, add-to-cart controls, search controls, order progress, and footer links.

Page-object method	Route hints	Assertion helpers used
verifyCART001EmptyCartPageOpens	"/cart"	verifyHeading, verifyTitle
verifyCART002EmptyCartMessageIsShown	"/cart"	verifyBodyText
verifyCART003HeaderShoppingCartLinkIsVisible	"/"	verifyBodyText, verifyVisible
verifyCART004CartFooterLinkIsVisible	"/"	verifyBodyText, verifyVisible
verifyCART005BookProductHasAddToCartControls	"/computing-and-internet"	verifyVisible
verifyCART006LaptopProductHasAddToCartControls	"/141-inch-laptop"	verifyVisible
verifyCART007ComputerProductHasAddToCartControls	"/build-your-own-computer"	verifyVisible
verifyCART008GiftCardProductHasAddToCartControls	"/25-virtual-gift-card"	verifyVisible
verifyCART009ApparelProductHasCartControls	"/blue-jeans"	verifyVisible
verifyCART010JewelryProductHasCartControls	"/create-it-yourself-jewelry"	verifyVisible
verifyCART011CartPageStillShowsSearchControls	"/cart"	verifyVisible
verifyCART012CartPageShowsOrderProgressSteps	"/cart"	verifyBodyText
verifyCART013CartPageShowsInformationFooterLinks	"/cart"	verifyBodyText
verifyCART014CartPageKeepsFooterFollowLinksAvailable	"/cart"	verifyBodyText, verifyVisible
verifyCART015CartPageHasCheckoutJourneyLabels	"/cart"	verifyBodyText

Wishlist and Compare

Wishlist page, compare products page, email-a-friend flow, recently viewed products, and footer links.

Page-object method	Route hints	Assertion helpers used
verifyWISH001WishlistPageOpens	"/wishlist"	verifyHeading, verifyTitle

verifyWISH002EmptyWishlistMessageIsShown	"/wishlist"	verifyBodyText
verifyWISH003HeaderWishlistLinkIsVisible	"/"	verifyBodyText, verifyVisible
verifyWISH004GiftCardProductHasWishlistButton	"/25-virtual-gift-card"	verifyVisible
verifyWISH005ComputerProductHasCompareButton	"/build-your-own-computer"	verifyVisible
verifyWISH006LaptopProductHasCompareButton	"/141-inch-laptop"	verifyVisible
verifyWISH007CompareProductsPageOpens	"/compareproducts"	verifyBodyText, verifyTitle
verifyWISH008RecentlyViewedProductsPageOpens	"/recentlyviewedproducts"	verifyBodyText, verifyTitle
verifyWISH009NewProductsPageOpens	"/newproducts"	verifyCountAtLeast, verifyTitle
verifyWISH010ProductPageExposesEmailAFriendAction	"/141-inch-laptop"	verifyVisible
verifyWISH011EmailAFriendPageOpens	"/productemailafriend/31"	verifyBodyText, verifyTitle
verifyWISH012EmailAFriendFormHasFriendEmailField	"/productemailafriend/31"	verifyVisible
verifyWISH013EmailAFriendFormHasMessageAndSendButton	"/productemailafriend/31"	verifyVisible
verifyWISH014CompareProductsFooterLinkIsAvailable	"/"	verifyBodyText, verifyVisible
verifyWISH015WishlistFooterLinkIsAvailable	"/"	verifyBodyText, verifyVisible

User Profile and Account

Guest access behavior for account pages, login/register account fields, footer account links, and sitemap account links.

Page-object method	Route hints	Assertion helpers used
verifyUSER001MyAccountPageRequiresLoginForGuest	"/customer/info"	verifyHeading
verifyUSER002OrderHistoryPageRequiresLoginForGuest	"/customer/orders"	verifyHeading
verifyUSER003AddressesPageRequiresLoginForGuest	"/customer/addresses"	verifyHeading
verifyUSER004LoginPageSupportsReturningCustomerSection	"/login"	verifyBodyText, verifyVisible
verifyUSER005FooterExposesMyAccountLink	"/"	verifyBodyText, verifyVisible
verifyUSER006FooterExposesOrdersLink	"/"	verifyBodyText, verifyVisible
verifyUSER007FooterExposesAddressesLink	"/"	verifyBodyText, verifyVisible
verifyUSER008RegisterPageShowsAccountCreationForm	"/register"	verifyHeading, verifyVisible
verifyUSER009RegisterPageShowsPersonalDetailsFields	"/register"	verifyVisible
verifyUSER010RegisterPageShowsEmailField	"/register"	verifyVisible
verifyUSER011RegisterPageShowsPasswordSetupFields	"/register"	verifyVisible
verifyUSER012LoginPageProvidesRegisterButtonForNewUser	"/login"	verifyBodyText, verifyVisible
verifyUSER013LoginPageProvidesForgotPasswordLink	"/login"	verifyBodyText
verifyUSER014PasswordRecoveryPageSupportsAccountRecovery	"/passwordrecovery"	verifyVisible
verifyUSER015SitemapIncludesAccountLinks	"/sitemap"	verifyBodyText

Address and Shipping

Shipping returns page, guest address redirect, product shipping messages, availability, cart address/shipping steps, and sitemap/footer shipping links.

Page-object method	Route hints	Assertion helpers used
verifyADDR001ShippingReturnsPageOpens	"/shipping-returns"	verifyHeading, verifyTitle
verifyADDR002ShippingReturnsPageHasCustomerServiceContext	"/shipping-returns"	verifyBodyText
verifyADDR003GuestAddressesPageRedirectsToLogin	"/customer/addresses"	verifyHeading
verifyADDR004BookProductDisplaysFreeShipping	"/computing-and-internet"	verifyBodyText
verifyADDR005ScienceProductDisplaysFreeShipping	"/science"	verifyBodyText
verifyADDR006HealthBookProductDisplaysDeliveryDate	"/health"	verifyBodyText
verifyADDR007LaptopProductShowsInStockAvailability	"/141-inch-laptop"	verifyBodyText
verifyADDR008SmartphoneProductShowsInStockAvailability	"/smartphone"	verifyBodyText
verifyADDR009BlueJeansProductShowsInStockAvailability	"/blue-jeans"	verifyBodyText
verifyADDR010ShoppingCartShowsAddressStep	"/cart"	verifyBodyText
verifyADDR011ShoppingCartShowsShippingStep	"/cart"	verifyBodyText
verifyADDR012FooterHasShippingReturnsLink	"/"	verifyBodyText, verifyVisible
verifyADDR013SitemapIncludesShippingReturnsLink	"/sitemap"	verifyBodyText, verifyVisible
verifyADDR014CartPageHasCheckoutProgressSection	"/cart"	verifyBodyText, verifyVisible
verifyADDR015ShippingPageKeepsInformationLinksAvailable	"/shipping-returns"	verifyBodyText

Add on - Checkout and Payment

Empty cart checkout behavior, checkout progress steps, gift card form fields, add-to-cart button, and product price checks.

Page-object method	Route hints	Assertion helpers used
verifyPAY001EmptyCartKeepsUserOnShoppingCartPage	"/cart"	verifyHeading
verifyPAY002EmptyCartBlocksCheckoutWithEmptyCartMessage	"/cart"	verifyBodyText
verifyPAY003CartPageIsAvailableBeforeCheckout	"/cart"	verifyBodyText
verifyPAY004CartPageShowsPaymentStep	"/cart"	verifyBodyText
verifyPAY005CartPageShowsConfirmStep	"/cart"	verifyBodyText
verifyPAY006CartPageShowsCompleteStep	"/cart"	verifyBodyText
verifyPAY007GiftCardProductHasRecipientNameField	"/25-virtual-gift-card"	verifyVisible
verifyPAY008GiftCardProductHasRecipientEmailField	"/25-virtual-gift-card"	verifyVisible
verifyPAY009GiftCardProductHasSenderFields	"/25-virtual-gift-card"	verifyVisible

verifyPAY010GiftCardProductHasMessageField	"/25-virtual-gift-card"	verifyVisible
verifyPAY011GiftCardProductHasAddToCartButton	"/25-virtual-gift-card"	verifyVisible
verifyPAY012GiftCardProductShowsFixedPrice	"/25-virtual-gift-card"	verifyBodyText
verifyPAY013BookProductShowsPriceInformation	"/computing-and-internet"	verifyBodyText
verifyPAY014LaptopProductShowsPriceInformation	"/141-inch-laptop"	verifyBodyText
verifyPAY015SmartphoneProductShowsPriceInformation	"/smartphone"	verifyBodyText

Add on - Customer Support and Information

Contact us page fields, sitemap, privacy, conditions, about, news, blog, footer links, newsletter, and social links.

Page-object method	Route hints	Assertion helpers used
verifyINFO001ContactUsPageOpens	"/contactus"	verifyHeading, verifyTitle
verifyINFO002ContactUsPageHasFullNameField	"/contactus"	verifyVisible
verifyINFO003ContactUsPageHasEmailField	"/contactus"	verifyVisible
verifyINFO004ContactUsPageHasEnquiryTextarea	"/contactus"	verifyVisible
verifyINFO005ContactUsPageHasSubmitButton	"/contactus"	verifyVisible
verifyINFO006SitemapPageOpens	"/sitemap"	verifyHeading, verifyTitle
verifyINFO007PrivacyNoticePageOpens	"/privacy-policy"	verifyHeading, verifyTitle
verifyINFO008ConditionsOfUsePageOpens	"/conditions-of-use"	verifyHeading, verifyTitle
verifyINFO009AboutUsPageOpens	"/about-us"	verifyHeading, verifyTitle
verifyINFO010NewsPageOpens	"/news"	verifyHeading, verifyTitle
verifyINFO011BlogPageOpens	"/blog"	verifyHeading, verifyTitle
verifyINFO012FooterInformationLinksAreVisible	"/"	verifyVisible
verifyINFO013FooterCustomerServiceLinksAreVisible	"/"	verifyVisible
verifyINFO014NewsletterSubscriptionControlsAreVisible	"/"	verifyVisible
verifyINFO015FooterSocialLinksAreVisible	"/"	verifyVisible

8. All 120 Functional Test Cases

Each row below is one functional test case. When all three browser projects are enabled, every row runs once in Chromium, once in Firefox, and once in WebKit.

Service	Functional test cases	Cross-browser executions
Authentication	15	45 executions
Product and Search	15	45 executions

Cart	15	45 executions
Wishlist and Compare	15	45 executions
User Profile and Account	15	45 executions
Address and Shipping	15	45 executions
Add on - Checkout and Payment	15	45 executions
Add on - Customer Support and Information	15	45 executions

How 120 becomes 360

The capstone count is 120 functional test cases. Playwright browser projects multiply execution count. 120 test cases x 3 browsers = 360 total browser executions in the Allure report.

Authentication

ID	Scenario	Page-object method	Spec file
AUTH-001	Login page opens with correct title and heading	verifyAUTH001LoginPageOpensWithCorrectTitleAndHeading	tests/authentication/auth-001-login-page-opens-with-correct-title-and-heading.spec.js
AUTH-002	Login page shows email and password fields	verifyAUTH002LoginPageShowsEmailAndPasswordFields	tests/authentication/auth-002-login-page-shows-email-and-password-fields.spec.js
AUTH-003	Login page shows login button	verifyAUTH003LoginPageShowsLoginButton	tests/authentication/auth-003-login-page-shows-login-button.spec.js
AUTH-004	Login page shows remember me and forgot password options	verifyAUTH004LoginPageShowsRememberMeAndForgotPasswordOptions	tests/authentication/auth-004-login-page-shows-remember-me-and-forgot-password-options.spec.js
AUTH-005	Login page shows registration option for new customer	verifyAUTH005LoginPageShowsRegistrationOptionForNewCustomer	tests/authentication/auth-005-login-page-shows-registration-option-for-new-customer.spec.js
AUTH-006	Password recovery page opens correctly	verifyAUTH006PasswordRecoveryPageOpensCorrectly	tests/authentication/auth-006-password-recovery-page-opens-correctly.spec.js
AUTH-007	Password recovery page has email field and recover button	verifyAUTH007PasswordRecoveryPageHasEmailFieldAndRecoverButton	tests/authentication/auth-007-password-recovery-page-has-email-field-and-recover-button.spec.js
AUTH-008	Register page opens correctly	verifyAUTH008RegisterPageOpensCorrectly	tests/authentication/auth-008-register-page-opens-correctly.spec.js
AUTH-009	Register page shows gender choices	verifyAUTH009RegisterPageShowsGenderChoices	tests/authentication/auth-009-register-page-shows-gender-choices.spec.js
AUTH-010	Register page shows personal detail fields	verifyAUTH010RegisterPageShowsPersonalDetailF	tests/authentication/auth-

		ields	010-register-page-shows-personal-detail-fields.spec.js
AUTH-011	Register page shows password fields	verifyAUTH011RegisterPageShowsPasswordFields	tests/authentication/auth-011-register-page-shows-password-fields.spec.js
AUTH-012	Register button is available	verifyAUTH012RegisterButtonIsAvailable	tests/authentication/auth-012-register-button-is-available.spec.js
AUTH-013	Protected account page redirects guest user to login	verifyAUTH013ProtectedAccountPageRedirectsGuestUserToLogin	tests/authentication/auth-013-protected-account-page-redirects-guest-user-to-login.spec.js
AUTH-014	Protected orders page redirects guest user to login	verifyAUTH014ProtectedOrdersPageRedirectsGuestUserToLogin	tests/authentication/auth-014-protected-orders-page-redirects-guest-user-to-login.spec.js
AUTH-015	Header exposes login and register links	verifyAUTH015HeaderExposesLoginAndRegisterLinks	tests/authentication/auth-015-header-exposes-login-and-register-links.spec.js

Product and Search

ID	Scenario	Page-object method	Spec file
PROD-001	Home page shows welcome content and featured products	verifyPROD001HomePageShowsWelcomeContentAndFeaturedProducts	tests/product/prod-001-home-page-shows-welcome-content-and-featured-products.spec.js
PROD-002	Top navigation exposes catalog categories	verifyPROD002TopNavigationExposesCatalogCategories	tests/product/prod-002-top-navigation-exposes-catalog-categories.spec.js
PROD-003	Books category shows product cards	verifyPROD003BooksCategoryShowsProductCards	tests/product/prod-003-books-category-shows-product-cards.spec.js
PROD-004	Computers category shows subcategories	verifyPROD004ComputersCategoryShowsSubcategories	tests/product/prod-004-computers-category-shows-subcategories.spec.js
PROD-005	Electronics category shows subcategories	verifyPROD005ElectronicsCategoryShowsSubcategories	tests/product/prod-005-electronics-category-shows-subcategories.spec.js
PROD-006	Apparel and shoes category shows products	verifyPROD006ApparelAndShoesCategoryShowsProducts	tests/product/prod-006-apparel-and-shoes-category-shows-products.spec.js
PROD-007	Digital downloads category opens	verifyPROD007DigitalDownloadsCategoryOpens	tests/product/prod-007-digital-downloads-category-opens.spec.js
PROD-008	Jewelry category opens with product cards	verifyPROD008JewelryCategoryOpensWithProductCards	tests/product/prod-008-jewelry-category-opens-with-product-cards.spec.js
PROD-009	Gift cards category opens with products	verifyPROD009GiftCardsCategoryOpensWithProducts	tests/product/prod-009-gift-cards-category-opens-with-products.spec.js

PROD-010	Build your own computer product details are visible	verifyPROD010BuildYourOwnComputerProductDetailsAreVisible	tests/product/prod-010-build-your-own-computer-product-details-are-visible.spec.js
PROD-011	Laptop product details are visible	verifyPROD011LaptopProductDetailsAreVisible	tests/product/prod-011-laptop-product-details-are-visible.spec.js
PROD-012	Smartphone product details are visible	verifyPROD012SmartphoneProductDetailsAreVisible	tests/product/prod-012-smartphone-product-details-are-visible.spec.js
PROD-013	Book product page shows price and shipping text	verifyPROD013BookProductPageShowsPriceAndShippingText	tests/product/prod-013-book-product-page-shows-price-and-shipping-text.spec.js
PROD-014	Category page has sort and page size controls	verifyPROD014CategoryPageHasSortAndPageSizeControls	tests/product/prod-014-category-page-has-sort-and-page-size-controls.spec.js
PROD-015	Search page returns computer products	verifyPROD015SearchPageReturnsComputerProducts	tests/product/prod-015-search-page-returns-computer-products.spec.js

Cart

ID	Scenario	Page-object method	Spec file
CART-001	Empty cart page opens	verifyCART001EmptyCartPageOpens	tests/cart/cart-001-empty-cart-page-opens.spec.js
CART-002	Empty cart message is shown	verifyCART002EmptyCartMessageIsShown	tests/cart/cart-002-empty-cart-message-is-shown.spec.js
CART-003	Header shopping cart link is visible	verifyCART003HeaderShoppingCartLinkIsVisible	tests/cart/cart-003-header-shopping-cart-link-is-visible.spec.js
CART-004	Cart footer link is visible	verifyCART004CartFooterLinkIsVisible	tests/cart/cart-004-cart-footer-link-is-visible.spec.js
CART-005	Book product has add to cart controls	verifyCART005BookProductHasAddToCartControls	tests/cart/cart-005-book-product-has-add-to-cart-controls.spec.js
CART-006	Laptop product has add to cart controls	verifyCART006LaptopProductHasAddToCartControls	tests/cart/cart-006-laptop-product-has-add-to-cart-controls.spec.js
CART-007	Computer product has add to cart controls	verifyCART007ComputerProductHasAddToCartControls	tests/cart/cart-007-computer-product-has-add-to-cart-controls.spec.js
CART-008	Gift card product has add to cart controls	verifyCART008GiftCardProductHasAddToCartControls	tests/cart/cart-008-gift-card-product-has-add-to-cart-controls.spec.js
CART-009	Apparel product has cart controls	verifyCART009ApparelProductHasCartControls	tests/cart/cart-009-apparel-product-has-cart-controls.spec.js
CART-010	Jewelry product has cart controls	verifyCART010JewelryProductHasCartControls	tests/cart/cart-010-jewelry-product-has-cart-controls.spec.js
CART-011	Cart page still shows search controls	verifyCART011CartPageStillShowsSearchControls	tests/cart/cart-011-cart-page-still-shows-search-

			controls.spec.js
CART-012	Cart page shows order progress steps	verifyCART012CartPageShowsOrderProgressSteps	tests/cart/cart-012-cart-page-shows-order-progress-steps.spec.js
CART-013	Cart page shows information footer links	verifyCART013CartPageShowsInformationFooterLinks	tests/cart/cart-013-cart-page-shows-information-footer-links.spec.js
CART-014	Cart page keeps footer follow links available	verifyCART014CartPageKeepsFooterFollowLinksAvailable	tests/cart/cart-014-cart-page-keeps-footer-follow-links-available.spec.js
CART-015	Cart page has checkout journey labels	verifyCART015CartPageHasCheckoutJourneyLabels	tests/cart/cart-015-cart-page-has-checkout-journey-labels.spec.js

Wishlist and Compare

ID	Scenario	Page-object method	Spec file
WISH-001	Wishlist page opens	verifyWISH001WishlistPageOpens	tests/wishlist-compare/wish-001-wishlist-page-opens.spec.js
WISH-002	Empty wishlist message is shown	verifyWISH002EmptyWishlistMessageIsShown	tests/wishlist-compare/wish-002-empty-wishlist-message-is-shown.spec.js
WISH-003	Header wishlist link is visible	verifyWISH003HeaderWishlistLinkIsVisible	tests/wishlist-compare/wish-003-header-wishlist-link-is-visible.spec.js
WISH-004	Gift card product has wishlist button	verifyWISH004GiftCardProductHasWishlistButton	tests/wishlist-compare/wish-004-gift-card-product-has-wishlist-button.spec.js
WISH-005	Computer product has compare button	verifyWISH005ComputerProductHasCompareButton	tests/wishlist-compare/wish-005-computer-product-has-compare-button.spec.js
WISH-006	Laptop product has compare button	verifyWISH006LaptopProductHasCompareButton	tests/wishlist-compare/wish-006-laptop-product-has-compare-button.spec.js
WISH-007	Compare products page opens	verifyWISH007CompareProductsPageOpens	tests/wishlist-compare/wish-007-compare-products-page-opens.spec.js
WISH-008	Recently viewed products page opens	verifyWISH008RecentlyViewedProductsPageOpens	tests/wishlist-compare/wish-008-recently-viewed-products-page-opens.spec.js
WISH-009	New products page opens	verifyWISH009NewProductsPageOpens	tests/wishlist-compare/wish-009-new-products-page-opens.spec.js
WISH-010	Product page exposes email a friend action	verifyWISH010ProductPageExposesEmailAFriendAction	tests/wishlist-compare/wish-010-product-page-exposes-email-a-friend-action.spec.js
WISH-011	Email a friend page opens	verifyWISH011EmailAFriendPageOpens	tests/wishlist-compare/wish-011-email-a-friend-page-opens.spec.js
WISH-012	Email a friend form has friend email field	verifyWISH012EmailAFriendFormHasFriendEmailF	tests/wishlist-compare/wish-

		ield	012-email-a-friend-form-has-friend-email-field.spec.js
WISH-013	Email a friend form has message and send button	verifyWISH013EmailAFriendFormHasMessageAndSendButton	tests/wishlist-compare/wish-013-email-a-friend-form-has-message-and-send-button.spec.js
WISH-014	Compare products footer link is available	verifyWISH014CompareProductsFooterLinkIsAvailable	tests/wishlist-compare/wish-014-compare-products-footer-link-is-available.spec.js
WISH-015	Wishlist footer link is available	verifyWISH015WishlistFooterLinkIsAvailable	tests/wishlist-compare/wish-015-wishlist-footer-link-is-available.spec.js

User Profile and Account

ID	Scenario	Page-object method	Spec file
USER-001	My account page requires login for guest	verifyUSER001MyAccountPageRequiresLoginForGuest	tests/user-profile/user-001-my-account-page-requires-login-for-guest.spec.js
USER-002	Order history page requires login for guest	verifyUSER002OrderHistoryPageRequiresLoginForGuest	tests/user-profile/user-002-order-history-page-requires-login-for-guest.spec.js
USER-003	Addresses page requires login for guest	verifyUSER003AddressesPageRequiresLoginForGuest	tests/user-profile/user-003-addresses-page-requires-login-for-guest.spec.js
USER-004	Login page supports returning customer section	verifyUSER004LoginPageSupportsReturningCustomerSection	tests/user-profile/user-004-login-page-supports-returning-customer-section.spec.js
USER-005	Footer exposes my account link	verifyUSER005FooterExposesMyAccountLink	tests/user-profile/user-005-footer-exposes-my-account-link.spec.js
USER-006	Footer exposes orders link	verifyUSER006FooterExposesOrdersLink	tests/user-profile/user-006-footer-exposes-orders-link.spec.js
USER-007	Footer exposes addresses link	verifyUSER007FooterExposesAddressesLink	tests/user-profile/user-007-footer-exposes-addresses-link.spec.js
USER-008	Register page shows account creation form	verifyUSER008RegisterPageShowsAccountCreationForm	tests/user-profile/user-008-register-page-shows-account-creation-form.spec.js
USER-009	Register page shows personal details fields	verifyUSER009RegisterPageShowsPersonalDetailsFields	tests/user-profile/user-009-register-page-shows-personal-details-fields.spec.js
USER-010	Register page shows email field	verifyUSER010RegisterPageShowsEmailField	tests/user-profile/user-010-register-page-shows-email-field.spec.js
USER-011	Register page shows password setup fields	verifyUSER011RegisterPageShowsPasswordSetupFields	tests/user-profile/user-011-register-page-shows-password-setup-fields.spec.js
USER-012	Login page provides register button for new user	verifyUSER012LoginPageProvidesRegisterButtonF	tests/user-profile/user-012-

		orNewUser	login-page-provides-register-button-for-new-user.spec.js
USER-013	Login page provides forgot password link	verifyUSER013LoginPageProvidesForgotPasswordLink	tests/user-profile/user-013-login-page-provides-forgot-password-link.spec.js
USER-014	Password recovery page supports account recovery	verifyUSER014PasswordRecoveryPageSupportsAccountRecovery	tests/user-profile/user-014-password-recovery-page-supports-account-recovery.spec.js
USER-015	Sitemap includes account links	verifyUSER015SitemapIncludesAccountLinks	tests/user-profile/user-015-sitemap-includes-account-links.spec.js

Address and Shipping

ID	Scenario	Page-object method	Spec file
ADDR-001	Shipping returns page opens	verifyADDR001ShippingReturnsPageOpens	tests/address-shipping/addr-001-shipping-returns-page-opens.spec.js
ADDR-002	Shipping returns page has customer service context	verifyADDR002ShippingReturnsPageHasCustomerServiceContext	tests/address-shipping/addr-002-shipping-returns-page-has-customer-service-context.spec.js
ADDR-003	Guest addresses page redirects to login	verifyADDR003GuestAddressesPageRedirectsToLogin	tests/address-shipping/addr-003-guest-addresses-page-redirects-to-login.spec.js
ADDR-004	Book product displays free shipping	verifyADDR004BookProductDisplaysFreeShipping	tests/address-shipping/addr-004-book-product-displays-free-shipping.spec.js
ADDR-005	Science product displays free shipping	verifyADDR005ScienceProductDisplaysFreeShipping	tests/address-shipping/addr-005-science-product-displays-free-shipping.spec.js
ADDR-006	Health book product displays delivery date	verifyADDR006HealthBookProductDisplaysDeliveryDate	tests/address-shipping/addr-006-health-book-product-displays-delivery-date.spec.js
ADDR-007	Laptop product shows in stock availability	verifyADDR007LaptopProductShowsInStockAvailability	tests/address-shipping/addr-007-laptop-product-shows-in-stock-availability.spec.js
ADDR-008	Smartphone product shows in stock availability	verifyADDR008SmartphoneProductShowsInStockAvailability	tests/address-shipping/addr-008-smartphone-product-shows-in-stock-availability.spec.js
ADDR-009	Blue jeans product shows in stock availability	verifyADDR009BlueJeansProductShowsInStockAvailability	tests/address-shipping/addr-009-blue-jeans-product-shows-in-stock-availability.spec.js
ADDR-010	Shopping cart shows address step	verifyADDR010ShoppingCartShowsAddressStep	tests/address-shipping/addr-010-shopping-cart-shows-address-step.spec.js
ADDR-011	Shopping cart shows shipping step	verifyADDR011ShoppingCartShowsShippingStep	tests/address-shipping/addr-011-shopping-cart-shows-shipping-step.spec.js

ADDR-012	Footer has shipping returns link	verifyADDR012FooterHasShippingReturnsLink	tests/address-shipping/addr-012-footer-has-shipping-returns-link.spec.js
ADDR-013	Sitemap includes shipping returns link	verifyADDR013SitemapIncludesShippingReturnsLink	tests/address-shipping/addr-013-sitemap-includes-shipping-returns-link.spec.js
ADDR-014	Cart page has checkout progress section	verifyADDR014CartPageHasCheckoutProgressSection	tests/address-shipping/addr-014-cart-page-has-checkout-progress-section.spec.js
ADDR-015	Shipping page keeps information links available	verifyADDR015ShippingPageKeepsInformationLinksAvailable	tests/address-shipping/addr-015-shipping-page-keeps-information-links-available.spec.js

Add on - Checkout and Payment

ID	Scenario	Page-object method	Spec file
Add on - PAY-001	Empty cart keeps user on shopping cart page	verifyPAY001EmptyCartKeepsUserOnShoppingCartPage	tests/add-on-checkout-payment/add-on-pay-001-empty-cart-keeps-user-on-shopping-cart-page.spec.js
Add on - PAY-002	Empty cart blocks checkout with empty cart message	verifyPAY002EmptyCartBlocksCheckoutWithEmptyCartMessage	tests/add-on-checkout-payment/add-on-pay-002-empty-cart-blocks-checkout-with-empty-cart-message.spec.js
Add on - PAY-003	Cart page is available before checkout	verifyPAY003CartPageIsAvailableBeforeCheckout	tests/add-on-checkout-payment/add-on-pay-003-cart-page-is-available-before-checkout.spec.js
Add on - PAY-004	Cart page shows payment step	verifyPAY004CartPageShowsPaymentStep	tests/add-on-checkout-payment/add-on-pay-004-cart-page-shows-payment-step.spec.js
Add on - PAY-005	Cart page shows confirm step	verifyPAY005CartPageShowsConfirmStep	tests/add-on-checkout-payment/add-on-pay-005-cart-page-shows-confirm-step.spec.js
Add on - PAY-006	Cart page shows complete step	verifyPAY006CartPageShowsCompleteStep	tests/add-on-checkout-payment/add-on-pay-006-cart-page-shows-complete-step.spec.js
Add on - PAY-007	Gift card product has recipient name field	verifyPAY007GiftCardProductHasRecipientNameField	tests/add-on-checkout-payment/add-on-pay-007-gift-card-product-has-recipient-name-field.spec.js
Add on - PAY-008	Gift card product has recipient email field	verifyPAY008GiftCardProductHasRecipientEmailField	tests/add-on-checkout-payment/add-on-pay-008-gift-card-product-has-recipient-email-field.spec.js
Add on - PAY-009	Gift card product has sender fields	verifyPAY009GiftCardProductHasSenderFields	tests/add-on-checkout-payment/add-on-pay-009-gift-card-product-has-sender-fields.spec.js

Add on - PAY-010	Gift card product has message field	verifyPAY010GiftCardProductHasMessageField	tests/add-on-checkout-payment/add-on-pay-010-gift-card-product-has-message-field.spec.js
Add on - PAY-011	Gift card product has add to cart button	verifyPAY011GiftCardProductHasAddToCartButton	tests/add-on-checkout-payment/add-on-pay-011-gift-card-product-has-add-to-cart-button.spec.js
Add on - PAY-012	Gift card product shows fixed price	verifyPAY012GiftCardProductShowsFixedPrice	tests/add-on-checkout-payment/add-on-pay-012-gift-card-product-shows-fixed-price.spec.js
Add on - PAY-013	Book product shows price information	verifyPAY013BookProductShowsPriceInformation	tests/add-on-checkout-payment/add-on-pay-013-book-product-shows-price-information.spec.js
Add on - PAY-014	Laptop product shows price information	verifyPAY014LaptopProductShowsPriceInformation	tests/add-on-checkout-payment/add-on-pay-014-laptop-product-shows-price-information.spec.js
Add on - PAY-015	Smartphone product shows price information	verifyPAY015SmartphoneProductShowsPriceInformation	tests/add-on-checkout-payment/add-on-pay-015-smartphone-product-shows-price-information.spec.js

Add on - Customer Support and Information

ID	Scenario	Page-object method	Spec file
Add on - INFO-001	Contact us page opens	verifyINFO001ContactUsPageOpens	tests/add-on-customer-support/add-on-info-001-contact-us-page-opens.spec.js
Add on - INFO-002	Contact us page has full name field	verifyINFO002ContactUsPageHasFullNameField	tests/add-on-customer-support/add-on-info-002-contact-us-page-has-full-name-field.spec.js
Add on - INFO-003	Contact us page has email field	verifyINFO003ContactUsPageHasEmailField	tests/add-on-customer-support/add-on-info-003-contact-us-page-has-email-field.spec.js
Add on - INFO-004	Contact us page has enquiry textarea	verifyINFO004ContactUsPageHasEnquiryTextarea	tests/add-on-customer-support/add-on-info-004-contact-us-page-has-enquiry-textarea.spec.js
Add on - INFO-005	Contact us page has submit button	verifyINFO005ContactUsPageHasSubmitButton	tests/add-on-customer-support/add-on-info-005-contact-us-page-has-submit-button.spec.js
Add on - INFO-006	Sitemap page opens	verifyINFO006SitemapPageOpens	tests/add-on-customer-support/add-on-info-006-sitemap-page-opens.spec.js
Add on - INFO-007	Privacy notice page opens	verifyINFO007PrivacyNoticePageOpens	tests/add-on-customer-support/add-on-info-007-privacy-notice-page-opens.spec.js

Add on - INFO-008	Conditions of use page opens	verifyINFO008ConditionsOfUsePageOpens	tests/add-on-customer-support/add-on-info-008-conditions-of-use-page-opens.spec.js
Add on - INFO-009	About us page opens	verifyINFO009AboutUsPageOpens	tests/add-on-customer-support/add-on-info-009-about-us-page-opens.spec.js
Add on - INFO-010	News page opens	verifyINFO010NewsPageOpens	tests/add-on-customer-support/add-on-info-010-news-page-opens.spec.js
Add on - INFO-011	Blog page opens	verifyINFO011BlogPageOpens	tests/add-on-customer-support/add-on-info-011-blog-page-opens.spec.js
Add on - INFO-012	Footer information links are visible	verifyINFO012FooterInformationLinksAreVisible	tests/add-on-customer-support/add-on-info-012-footer-information-links-are-visible.spec.js
Add on - INFO-013	Footer customer service links are visible	verifyINFO013FooterCustomerServiceLinksAreVisible	tests/add-on-customer-support/add-on-info-013-footer-customer-service-links-are-visible.spec.js
Add on - INFO-014	Newsletter subscription controls are visible	verifyINFO014NewsletterSubscriptionControlsAreVisible	tests/add-on-customer-support/add-on-info-014-newsletter-subscription-controls-are-visible.spec.js
Add on - INFO-015	Footer social links are visible	verifyINFO015FooterSocialLinksAreVisible	tests/add-on-customer-support/add-on-info-015-footer-social-links-are-visible.spec.js

9. Reports, Allure, and GitHub Pages

A test suite is not useful only because it runs. It must also produce evidence. This project creates two kinds of local evidence and one hosted report.

Report/output	Created by	Purpose
playwright-report/	HTML reporter	Local Playwright report with test status, duration, traces, and attachments.
allure-results/	allure-playwright reporter	Raw JSON result files used by Allure.
allure-report/	allure generate command	Static HTML report that can be hosted.
GitHub Pages	GitHub Actions deployment	Trainer can open report in browser without running code locally.

Allure Workflow Logic

12. GitHub Actions starts on push to main/master or manual workflow_dispatch.
13. It downloads the repository source for the exact commit.

14. It runs npm ci to install exact dependencies from package-lock.json.
15. It installs Playwright browser dependencies on the Ubuntu runner.
16. It runs npx playwright test.
17. It checks that Allure has exactly 360 passed result files.
18. It generates allure-report from allure-results.
19. It force-pushes the generated report to the gh-pages branch.
20. GitHub Pages serves the gh-pages branch as the public report site.

```
name: Playwright Allure Report

on:
  push:
    branches: [ main, master ]
  workflow_dispatch:

permissions:
  contents: write

concurrency:
  group: allure-pages
  cancel-in-progress: false

jobs:
  tests:
    name: Run Tests and Publish Allure Report
    runs-on: ubuntu-latest
    timeout-minutes: 45
    steps:
      - name: Download repository
        run: |
          curl -L "https://codeload.github.com/${GITHUB_REPOSITORY}/tar.gz/${GITHUB_SHA}" -o
source.tar.gz
          tar -xzf source.tar.gz --strip-components=1
          node --version
          npm --version

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright Browsers
        run: npx playwright install --with-deps

      - name: Run Playwright Tests
        continue-on-error: true
        run: npx playwright test

      - name: Verify Playwright Results
        run: |
```

```
          node - <<'NODE'
          const fs = require('fs');
          const path = require('path');
          const resultsDir = path.join(process.cwd(), 'allure-results');
          const resultFiles = fs.existsSync(resultsDir)
            ? fs.readdirSync(resultsDir).filter((file) => file.endsWith('-result.json'))
            : [];

          const results = resultFiles.map((file) => {
            const resultPath = path.join(resultsDir, file);
            return JSON.parse(fs.readFileSync(resultPath, 'utf8'));
          });

          const nonPassed = results.filter((result) => result.status !== 'passed');
          console.log(`Allure result files: ${results.length}`);
          console.log(`Passed: ${results.filter((result) => result.status === 'passed').length}`);
          console.log(`Non-passed: ${nonPassed.length}`);

          if (results.length !== 360 || nonPassed.length > 0) {
            console.error('Expected exactly 360 passed executions: 120 tests on Chromium, Firefox, and
WebKit.');
            process.exit(1);
          }
          NODE

      - name: Install Allure Commandline
        if: always()
        run: npm install -g allure-commandline --save-dev

      - name: Generate Allure report
        if: always()
        run: allure generate allure-results --clean -o allure-report

      - name: Deploy Report to GitHub Pages
        if: success()
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          git config --global user.name "github-actions[bot]"
          git config --global user.email "41898282+github-actions[bot]@users.noreply.github.com"
          rm -rf gh-pages-publish
          mkdir gh-pages-publish
          cp -R allure-report/. gh-pages-publish/
          touch gh-pages-publish/.nojekyll
          cd gh-pages-publish
          git init
          git checkout -b gh-pages
...remaining lines are same project pattern...
```

Why generated report folders are normally ignored

Folders like allure-report, playwright-report, test-results, and node_modules are generated output. A clean repository should keep source code and configuration, then regenerate reports through commands or CI/CD.

10. Debugging and Troubleshooting

Problem	Reason	Fix
npm install undefined@node_modules error	The text after npm install was treated as package names.	Run only npm install from the project root.
0 tests found	Wrong folder or testDir mismatch.	Open D:\WIPRO\Capstone_Project and run npx playwright test --list.
Only Chromium shown in report	Only one project was run.	Run npx playwright test without --project, or check projects in config.
360 tests skipped	Command or config filtered browsers/tests incorrectly, or tests were skipped by grep/project selection.	Use npx playwright test --list, then run without grep and without --project.
Allure report shows 404 tile	A static Allure report may request optional widgets/history files that are absent.	If overview and suites load, the report is still usable; history can be added later.
GitHub Pages 404	gh-pages branch missing or Pages source not set to gh-pages root.	Run workflow successfully, then set Pages to Deploy from branch, gh-pages, root.
Cloudflare verification on old site	The website blocked automation in CI/browser.	Project moved to Demo Web Shop to avoid Cloudflare verification.
Locator timeout	Element not found, page slow, or wrong selector.	Check headed mode, trace, screenshot, and selector in browser devtools.
Tests pass locally but fail in GitHub Actions	CI is Linux, slower, and has different browser environment.	Use stable waits through expect, avoid hard timeouts, and check action logs.

Debugging Commands

```
npx playwright test --list
npx playwright test --project=chromium --headed
npx playwright test tests/authentication --project=chromium
npx playwright test --debug
npx playwright show-report
npx allure generate allure-results --clean -o allure-report
npx allure open allure-report
```

11. How to Add a New Test Professionally

21. Decide the service. Example: Product and Search.
22. Create a clear test ID. Example: PROD-016.
23. Add a method in the matching page object, such as ProductPage.
24. Inside that method, navigate to the page and write assertions using BasePage helpers.
25. Create a new .spec.js file in the service folder.
26. Import the page object and call only that one method from the test.
27. Run the service folder first, then the full suite.
28. Check Playwright report and Allure report.

Spec File Template

```
// @ts-check
import { test } from '@playwright/test';
import { ProductPage } from '../../pages/product.page.js';


test.describe('Product and Search Module', () => {
  let productPage;


  test.beforeEach(async ({ page }) => {
    productPage = new ProductPage(page);
  });


  test('PROD-016 New scenario name', async () => {
    await productPage.verifyPROD016NewScenarioName();
  });
});
```

Page Object Method Template

```
async verifyPROD016NewScenarioName() {
  await this.navigate('/some-route');
  await this.verifyHeading('Expected heading');
  await this.verifyVisible('.expected-selector');
}
```

12. Viva Preparation

These answers are written in the way you can speak during viva. Learn the idea first; do not just memorize words.

Q: What is your project about?

A: It is a Playwright JavaScript automation project for Demo Web Shop. I automated 120 functional test cases across 6 core modules and 2 add-on modules and configured them to run on Chromium, Firefox, and WebKit with Allure reporting.

Q: Why did you choose Demo Web Shop?

A: It is a stable e-commerce website with enough real modules like login, products, cart, wishlist, account, shipping, checkout-related pages, and customer support pages. It does not create Cloudflare automation problems.

Q: What is Playwright?

A: Playwright is a browser automation and testing framework. It can launch browsers, interact with pages, assert expected behavior, and generate reports.

Q: Which language did you use?

A: JavaScript with ES module import/export syntax.

Q: What is the role of package.json?

A: It stores project metadata, npm scripts, and dependencies such as Playwright and Allure packages.

Q: What is the role of package-lock.json?

A: It locks exact dependency versions so the same packages install on another machine or in GitHub Actions.

Q: What is playwright.config.js?

A: It is the central Playwright configuration file. It defines testDir, browsers, reporters, baseURL, timeouts, retries, and artifact settings.

Q: Why are there 360 executions in report?

A: Because there are 120 functional test cases and the config runs them on 3 browser projects: Chromium, Firefox, and WebKit.

Q: What is Page Object Model?

A: It is a design pattern where page-specific actions and assertions are kept in page classes, while spec files remain clean and scenario-focused.

Q: Why did you create BasePage?

A: BasePage stores common helper methods like navigate, verifyTitle, verifyVisible, verifyEnabled, and verifyBodyText so all page classes reuse the same logic.

Q: What is a spec file?

A: A spec file is a Playwright test file. In this project each spec contains one named test case that calls a page-object method.

Q: What does test.describe do?

A: It groups related tests under one module name in the report.

Q: What does test.beforeEach do?

A: It runs before each test. Here it creates a fresh page object using the current Playwright page.

Q: What is page in Playwright?

A: page represents one browser tab controlled by Playwright.

Q: What is locator?

A: A locator is Playwright's way of finding an element. It is auto-waiting, so Playwright waits for the element condition before acting or asserting.

Q: What is expect?

A: expect is used for assertions. If an expected condition is false, the test fails.

Q: Why not use hard waits?

A: Hard waits make tests slow and flaky. Playwright assertions auto-wait, so expect(locator).toBeVisible() is better.

Q: What reports are generated?

A: Playwright HTML report and Allure report. Allure is also deployed to GitHub Pages.

Q: What is allure-results?

A: It contains raw JSON result files produced by the allure-playwright reporter.

Q: What is allure-report?

A: It is the generated static HTML report created from allure-results.

Q: What is GitHub Actions?

A: It is CI/CD automation in GitHub. It runs the tests and deploys the report when code is pushed.

Q: What is gh-pages branch?

A: It is the branch used by GitHub Pages to host the generated Allure report.

Q: Why are generated folders ignored?

A: They are output, not source code. They can be regenerated by commands or CI.

Q: How do you run all tests locally?

A: From D:\WIPRO\Capstone_Project, run npx playwright test.

Q: How do you run one service?

A: Run npx playwright test tests/authentication or replace authentication with another service folder.

Q: How do you run one browser?

A: Run npx playwright test --project=chromium, or use firefox/webkit.

Q: How do you debug?

A: Run in headed mode or debug mode, then check traces, screenshots, HTML report, and terminal errors.

Q: What happens if one assertion fails?

A: That test fails, Playwright records failure information, and the report shows failed status with details.

Q: What are retries?

A: Retries rerun failed tests in CI once, which helps against temporary network or timing issues.

Q: Why workers: 1?

A: It keeps execution stable and easy to debug for this capstone website.

Q: What is baseURL?

A: It is the common website URL. Using baseURL lets us navigate with relative routes like /login.

Q: What is trace?

A: Trace is a Playwright artifact that records actions, screenshots, and DOM snapshots for debugging failed tests.

Q: What is screenshot only-on-failure?

A: Playwright captures screenshot only when a test fails, keeping reports clean.

Q: What is CI/CD in your project?

A: CI runs tests automatically and CD deploys the generated Allure report to GitHub Pages.

Q: What is the most important benefit of your structure?

A: It is easy to explain, maintain, and expand because tests, page objects, config, data, and reports have separate responsibilities.

Q: How will you add more tests?

A: Add a page-object method, add a spec file that calls it, run the service, then run the full suite and check reports.

Q: What are your 6 core modules and 2 add-on modules?

A: Authentication, Product and Search, Cart, Wishlist and Compare, User Profile and Account, Address and Shipping, Add on - Checkout and Payment, and Add on - Customer Support and Information.

Q: What did you learn from this project?

A: I learned how to structure a Playwright project, write stable assertions, use POM, run tests across browsers, generate Allure reports, and deploy reports with GitHub Actions.

13. Final Checklist Before Submission

- All 6 core module folders and 2 add-on module folders are present under tests/.
- Each service contains 15 spec files.
- Total functional tests are 120.
- playwright.config.js contains chromium, firefox, and webkit projects.
- npx playwright test --list shows tests under all browser projects.
- Allure reporter is configured.
- allure-results and allure-report are generated outputs.
- GitHub Actions workflow can run npm ci, install browsers, run tests, generate Allure, and deploy gh-pages.
- GitHub Pages source is gh-pages branch root.
- README explains the project without unnecessary personal notes.
- Documents in docs/ are current: testing plan, final report, and Playwright notes.

Final confidence line

If you can explain the command, the config, one spec file, one page-object method, BasePage, Allure, and GitHub Actions, you can handle most viva questions for this project.